

cmta.

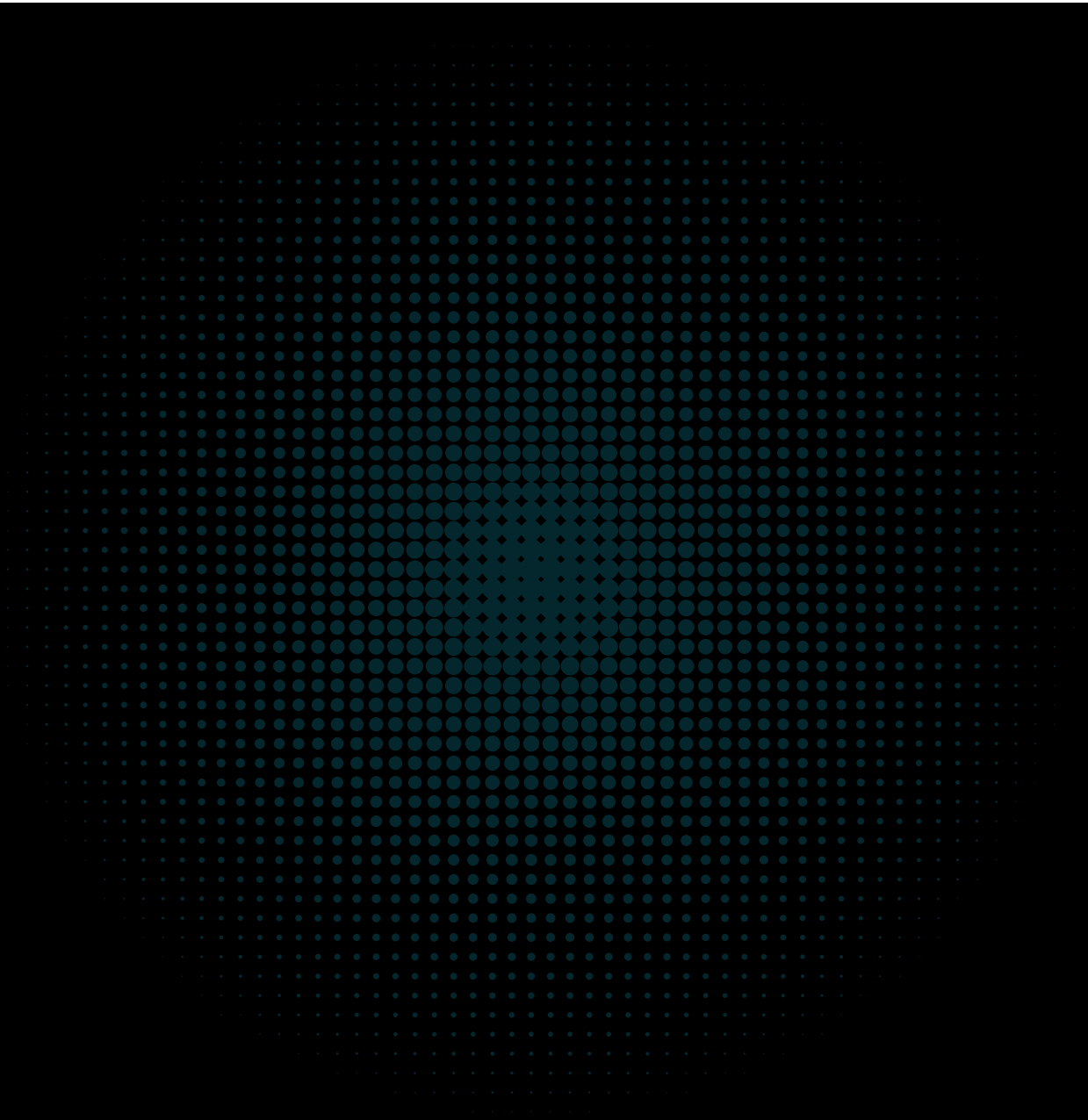
CMTAT Aztec Assessment

**Assessment of CMTAT on Aztec (Ethereum L2)
to enable confidential balances and transfers.**

CMTAT Private Aztec Assessment version: v0.1.0

CMTAT Private Aztec: v0.5.0

First published: October 2026



CMTAT Equivalency Assessment — private CMTAT on Aztec

This document is a **filled copy** of the [CMTAT Equivalency Assessment Criteria](#) template, completed for the Aztec implementation in this repository. Columns 1–5 of every criteria table are the template's pre-filled reference data about CMTAT Solidity and **MUST NOT** be modified; columns 6–8 are the answers for this implementation.

This implementation has not been audited. The answers below describe what the code does, not an assurance that it does so correctly.

Table of Contents

- [Document Version](#)
- [Metadata](#)
- [Architecture](#)
 - [Underlying ledger](#)
 - [Smart contract layer](#)
 - [Layer of each CMTAT module](#)
- [Summary](#)
 - [Scope of the count](#)
 - [Answer values](#)
 - [Compliance table](#)
- [CMTAT Function Equivalency Table](#)
 - [Token Attributes](#)
 - [Token module](#)
 - [Pause module \(mandatory\)](#)
 - [Enforcement](#)
 - [Transfer restriction \(optional\)](#)
 - [Access Control](#)
 - [Snapshot \(optional\)](#)
 - [Dividend \(optional\)](#)
 - [Credit Events \(optional\)](#)
 - [Debt \(optional\)](#)
- [Guideline sections](#)
 - [Freeze](#)
 - [Restriction \(optional\)](#)
 - [Version](#)
 - [CMTAT Extended](#)
 - [Forced Burn and Forced Transfer](#)
 - [Implementation Details](#)
 - [Self-Burn](#)
 - [Cross-Chain Bridge Support](#)
 - [Privacy and Confidentiality](#)
- [Supplementary features](#)

- [Conclusion](#)
- [Reference](#)

Document Version

Version	Value
Template version — this document, as published by CMTA; pre-filled, MUST NOT be modified by the author of an assessment	v0.4.0
Assessment version — the filled document, set by its author	0.1.0

The two numbers are independent, and the assessment's own version had previously been set to mirror the template's, which is what the template forbids. This is the first published revision of the filled assessment, so it is 0.1.0; earlier drafts numbered 0.2.0 and 0.3.0 were never published. Being below 1.0 it remains a **draft**, filled against an implementation that is itself a prototype and has not been audited.

Metadata

Field	Value
Implementation name	private CMTAT on Aztec — variants CMTATAztec (base), CMTATAztecDebt, CMTATAztecLight
Target blockchain or distributed ledger	Aztec (privacy L2 on Ethereum)
Implementation language	Noir / Aztec.nr v5.2.0
Implementation version	0.5.0, as returned by version() — see criterion 6
Source repository and commit	https://github.com/CMTA/private-CMTAT-aztec — 3843a36eedd91d2ac83d04e142b1e29536bfb48c. 0.5.0 is not tagged yet, so the commit is the reference. Nothing in contracts/, lib/ or test-helpers/ differs from the v0.4.0 tag except the VERSION constant, so the answers below are unchanged by the bump.
Assessment date	2026-10-05
Assessed by	(to be completed by the assessor)

Part of this document has been written with [Claude Code](#) by Anthropic.

Deployment variants

Noir has no inheritance and allows one contract per package, so the CMTAT variants are separate contract packages composing a shared module library, rather than a base contract with mixins. This assessment answers for `CMTATAztec`, the base variant, and marks the criteria that only a different variant satisfies.

	<code>CMTATAztecLight</code>	<code>CMTATAztec</code>	<code>CMTATAztecDebt</code>
Private mint / transfer / burn, issuer audit copies	✓	✓	✓
Pause, deactivation, freeze	✓	✓	✓
Access control, terms, token ID, version	✓	✓	✓
Validation module (blacklist / whitelist)	✗	✓	✓
Credit events (criteria 44–47)	✗	✗	✓
Debt (criteria 48–61)	✗	✗	✓

Criteria 44–61 are answered `y` below because the feature exists in the implementation, in the variant built to carry it; an assessment of `CMTATAztec` or `CMTATAztecLight` alone would answer them `n`. An issuer deploying a bond deploys `CMTATAztecDebt`, exactly as a CMTAT Solidity issuer deploys CMTAT Debt rather than CMTAT Standard.

Architecture

This section describes the platform the implementation runs on, so that the answers below can be read without prior knowledge of Aztec. It is context for the criteria and is **outside the equivalency count**: the 61 criteria and their IDs are unchanged.

Underlying ledger

General description

[Aztec](#) is a privacy-focused Layer 2 that settles to Ethereum. Its distinguishing property is that a transaction has two halves. The **private** half is executed and proved on the holder's own device, inside a zero-knowledge proof, by a local component called the PXE (private execution environment); what reaches the chain from it is commitments — note hashes and nullifiers — never the data itself. The **public** half is executed by the sequencer in the open, exactly as an EVM contract would be. A single transfer normally uses both: the value moves privately, and a public call is enqueued to check a flag or update a published counter.

The state model differs from an EVM ledger in the way that matters most for these criteria. Private state is **UTXO-like**: a holder's balance is a set of *notes*, each a commitment in a Merkle tree, spent by publishing a nullifier derived from the note and its owner's key. The notes themselves live in the holder's PXE, not in contract storage, and **the contract cannot read them**. Public state is ordinary key-value contract storage. Every account is itself a contract (native account abstraction), and an address is a point on the Grumpkin curve, so roughly half of all field values are not valid addresses.

The path of a private transfer is therefore: the holder's PXE selects enough of its own notes, proves the transfer circuit locally, publishes the nullifiers of the spent notes and the commitments of the new ones, and enqueues the public half; the sequencer runs that public half, which can revert and take the whole transaction with it.

Two consequences run through the whole assessment:

- An issuer **cannot move or destroy a holder's notes**, because nullifying one requires that holder's key. This is why forced transfer, forced burn and partial freeze are answered `n` and cannot be implemented at all.
- A private function **cannot read ordinary public state**, because its proof is built against a historical state. Values a private function must consult are held in `DelayedPublicMutable`, whose writes take effect only after a delay.

Details

Aspect	Implementation being approved
Ledger type and governance	Public, permissionless Layer 2 settling to Ethereum. Blocks are proposed by a sequencer drawn from a validator set and proven to L1. A mainnet exists — "Alpha", on L1 chain ID 1 — alongside a Sepolia-backed testnet. Both ran protocol version 5.1.0 when this was checked, while this implementation targets 5.2.0 , so it is not deployable to either as pinned; the network documentation also directs builders to validate on testnet before Alpha.
Consensus and finality	Proposer-based block production with proofs settled on Ethereum L1; finality follows L1 once the enclosing epoch is proven. A transaction is also bounded by an <code>expiration_timestamp</code> , and the protocol caps inclusion at 24 hours.
State model	Hybrid. Private state is UTXO-like notes committed to a Merkle tree with a nullifier set; public state is key-value contract storage. A holder's private balance is the sum of its <code>UintNotes</code> and is known only to that holder and to the issuer, which receives a copy of each note.
Native asset support	No native asset primitive for application tokens — a token is a contract. The ledger's only native asset is Fee Juice , used to pay fees, bridged from Ethereum, held as a public balance and non-transferable.
Native compliance features	None. The ledger offers no freeze, clawback, authorization flag or transfer hook. Every control in this assessment is contract logic.
Identity and addresses	Every account is a contract (native account abstraction), so there is no externally-owned-account equivalent. An address is derived from the contract class and its initialisation, and must be a valid point on the Grumpkin curve. An account must be deployed before it can send a transaction, but it can receive notes before deployment.

Aspect	Implementation being approved
Transactions and fees	A transaction runs in phases (setup, application logic, teardown), private before public. The fee payer is chosen per transaction , so a holder need not hold Fee Juice: a fee-paying contract may pay instead. Per-call protocol limits on note reads, nested private calls and logs are what cap this token's batch sizes.
Time source	Block timestamp, in seconds . The <code>DelayedPublicMutable</code> delays used for the freeze, list and issuer values are durations in seconds, not block counts.
Data visibility	Mixed by design, and the subject of Privacy and Confidentiality . Private balances and the parties and amounts of a private transfer are confidential. Public: <code>total_supply</code> , roles, the pause and deactivation flags, the freeze and list flags per address, and the fact that a transfer of this token occurred. A private function that reads a delayed value also publishes the transaction's expiration timestamp.

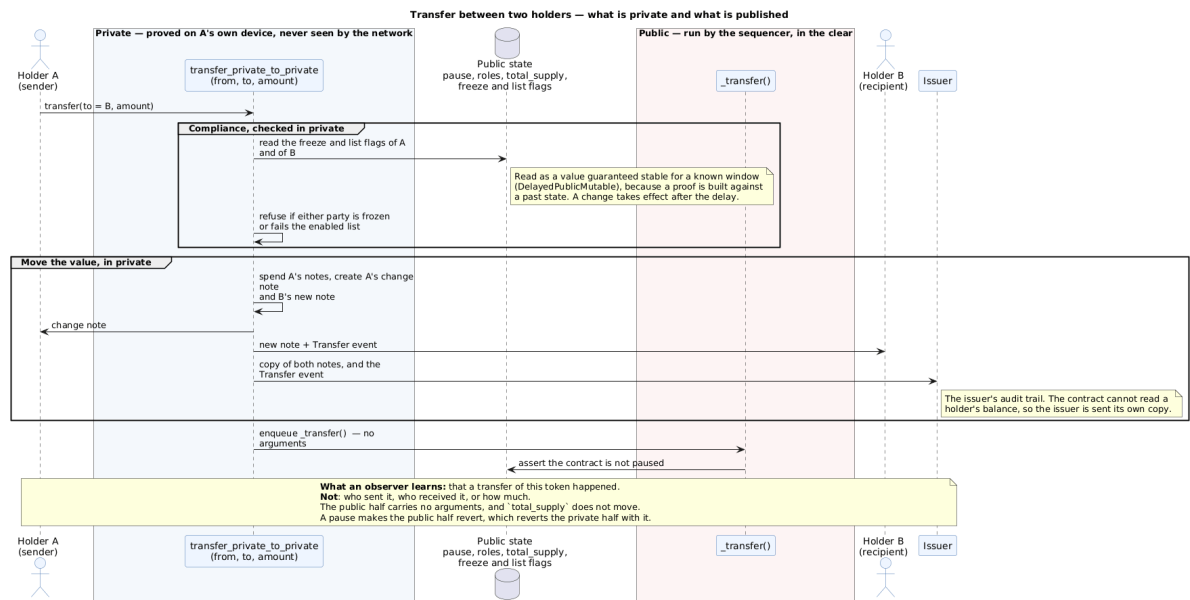
Smart contract layer

General description

The token is a smart contract, written in Noir with the Aztec.nr framework at v5.2.0. Private functions are compiled to zero-knowledge circuits and proved on the holder's device; public functions are transpiled to bytecode for the Aztec Virtual Machine and executed by the sequencer. The two are written in the same source file and the same language, and a private function reaches its public counterpart by *enqueueing* a call, which runs after the whole private part has completed.

Noir has no inheritance and allows one contract per crate, so the structure differs from CMTAT Solidity, where one contract inherits a stack of modules. Here the CMTAT modules are plain structs held as fields of a single `#[storage]` struct, and the shared logic lives in a library crate that the contracts compose. That is why the repository ships **five contracts over one library**: three token variants (base, Debt, Light) and two ARC-403 authorization contracts, which are not tokens but apply this token's controls to a third-party standard token through the hook it already calls.

A call made by a holder goes through the same chain as CMTAT Solidity's, split across the two halves: the **private** half screens both parties' freeze flags and the enabled list and moves the notes, then the **public** half it enqueues performs the role check and the lifecycle check. The split is forced by the ledger rather than chosen: roles and the pause flag are public state, which a private function cannot read at proving time. A revert in the public half reverts the entire transaction, so the ordering costs no safety.



Details

Aspect	Implementation being approved
Execution environment	Noir / Aztec.nr v5.2.0 . Private functions compile to zero-knowledge circuits; public functions are transpiled to AVM bytecode.
Components	Five contract crates over one library crate (<code>cmtat_aztec_lib</code>): <code>CMTATAztec</code> (base), <code>CMTATAztecDebt</code> , <code>CMTATAztecLight</code> , and the two authorization contracts <code>CMTATAztecAuth</code> / <code>CMTATAztecAuthMultiToken</code> . The CMTAT modules are structs inside the library, composed into each contract's single storage struct; there is no inheritance and no cross-contract call between the token's own parts.
Token state location	Split. Private balances are notes held in each holder's PXE, addressed by a storage slot in the contract; <code>total_supply</code> , roles, pause and deactivation flags, freeze and list flags, terms, token id, credit events and debt are public contract storage. Public balances exist only when the deployment enables them.
External dependencies	The <code>aztec-nr</code> libraries at v5.2.0 (<code>aztec</code> , <code>uint_note</code> , <code>balance_set</code> , <code>compressed_string</code>) and nothing else: no rule engine, no snapshot engine, no oracle and no bridge . The two authorization contracts additionally target the <code>aztec-standards</code> token contracts.
Deployment and ownership	A public constructor takes the issuer address, name, symbol, decimals and a flag enabling the public-balance paths. It grants <code>DEFAULT_ADMIN_ROLE</code> to the deployer-designated admin and schedules the issuer address, which becomes readable only after the delay. Administrative rights move by granting and revoking roles; there is no two-step ownership handover.

Aspect	Implementation being approved
Upgradeability	None, and not retrofittable. There is no proxy and no native upgrade path in use. A change of logic means a new deployment and a migration — and the migration cannot be performed by the issuer alone, because private balances are notes in holders' PXEs rather than storage the issuer can read and rewrite. This is why any storage or note-layout change is treated as MAJOR in the project's versioning policy.

Layer of each CMTAT module

CMTAT module	CMTAT Solidity	Layer (implementation being approved)	Comment
Token attributes	<code>contract</code>	<code>contract</code>	Name, symbol and decimals as compile-time values; terms and token id in public storage.
Mint and burn	<code>contract</code>	<code>contract</code>	Private note movement plus an enqueued public supply update carrying the role and lifecycle checks.
Pause and deactivation	<code>contract</code>	<code>contract</code>	Public flags. The pause is deliberately not delayed, so that it remains an immediate lever.
Enforcement (freeze, partial freeze, forced transfer)	<code>contract</code>	<code>contract</code> — freeze only	Freeze is a per-address delayed public flag. Partial freeze and forced transfer are not implementable : spending a note requires its owner's key.
Transfer restriction	<code>contract</code> — external rule engine	<code>contract</code> — integrated, not external	The validation module with its blacklist and whitelist is part of the token, not a separate pluggable engine; there is no <code>RuleEngine</code> equivalent and no sanction-list oracle, the ledger offering no register to read.
Access control	<code>contract</code>	<code>contract</code>	Eleven numeric roles in public state, modelled on OpenZeppelin <code>AccessControl</code> v5, with a fixed one-level hierarchy.

CMTAT module	CMTAT Solidity	Layer (implementation being approved)	Comment
Snapshot	<code>contract</code> — external snapshot engine	<code>none</code>	Not implementable as designed: balances are notes in holders' PXEs, so no vantage point can enumerate holders or sum balances at a past block. An issuer could reconstruct one off-chain from its note copies.
Dividend	<code>none</code> — prototype in IncomeVault	<code>none</code>	Absent, as in CMTAT Solidity.
Credit events and debt	<code>contract</code>	<code>contract</code> — CMTATAztecDebt only	Packed structs in public state, written by role-restricted setters; the base and Light variants do not carry them.

Note

Two architectural points are worth stating alongside the tables, because they explain answers that would otherwise look like omissions.

The issuer's audit trail is a design obligation, not a by-product. Because the contract cannot read a holder's notes, the issuer is given its own copy of every note created, plus a `Transfer` event delivered under proof for every movement. That event stream, rather than contract storage, is what lets an issuer reconstruct holdings — and it is why this token can answer the transparency criteria at all.

Three controls are impossible rather than unimplemented. Forced transfer, forced burn and partial freeze each require the issuer to spend or restrict a holder's notes without that holder's key. No role and no contract change can grant that, so these are answered `n` with "cryptographically impossible" rather than "not implemented", and the compliance workaround is to freeze the account and, if the freeze is permanent, to reduce the circulating supply accordingly.

Summary

Scope of the count

Category	Count	IDs
Mandatory	19	1-3, 7-11, 14-21, 29-31
Optional	42	4-6, 12-13, 22-28, 32-61

Answer values

Value	Meaning
y	Present — an equivalent feature exists and covers the requirement, even if the name, the signature, or the chain-level mechanism differs from CMTAT Solidity.
partial	Partial — an equivalent feature exists but covers only a part of the requirement, or covers it with a restriction, a different access control model, or different semantics.
n	Absent — no equivalent feature is available in the implementation being approved.

Compliance table

Answer	Mandatory (19)	Optional (42)
Present (y)	19	23
Partial	0	1
Absent (n)	0	18

Every mandatory criterion is answered y. Under the rule stated in this template, the implementation should be considered equivalent to CMTAT: no mandatory criterion is answered n, and none is answered partial.

Three criteria answered n in earlier revisions of this assessment have since been implemented: 17 and 18 (deactivation and its status) and 2 (reference to legally required documentation). Freeze and unfreeze (19 and 20) were answered partial in those revisions because of the delay before a freeze becomes effective; they are answered y here, for the reason given under [Enforcement](#).

Two cautions belong with that result. The implementation has **not been audited**, and equivalency to CMTAT is not by itself a demonstration that the criteria required for tokenized shares under Swiss law are satisfied — see the template's own warning on that point. Separately, the optional criteria that remain absent include forced transfer and forced burn, which several jurisdictions expect of a security token; a reader should not take "equivalent" to mean the token carries every regulatory recovery power.

Note

Optional partial answer

Criterion 13 (Approve) — Partial: delegation exists, but as an Aztec authentication witness rather than a standing ERC-20 allowance. An authwit authorises one exact call (target, selector, arguments and nonce), is consumed by a nullifier on use, and can be revoked before use with `cancel_authwit`. It therefore covers delegated spending, which is what the criterion is for, but it cannot express "this spender may move up to X over time": a new witness is required per operation. Secondary-market flows that assume a persistent allowance would need adapting.

Optional modules left out by design

Snapshot (criteria 32–37) and Dividend (criteria 38–43) are absent as whole modules — 12 of the 18 optional `n` answers. Snapshot in particular is not merely unimplemented: balances are UTXO notes held in each holder's own PXE, and there is no vantage point from which the contract can enumerate holders or sum balances at a past block. A snapshot would have to be reconstructed off-chain by the issuer from its copies of the notes.

Forced transfer (criterion 22) and the partial-freeze family (criteria 23–25) are absent because spending a note requires its owner's nullifier key, so the issuer cannot move or lock another holder's tokens — a different cause from the delay behind criteria 19 and 20. See [Forced Burn and Forced Transfer](#).

Conditional transfer (criteria 26–27) is absent; the validation module offers list-based restriction only.

All fourteen debt criteria (48–61) are now answered `y`. The debt structs were realigned with the CMTAT Solidity `ICMTATDebt` interface, which added the fields behind criteria 52 (currency of payments) and 54 (minimum denomination); criterion 50 was already covered by `token_id()` and the terms document hash.

CMTAT Function Equivalency Table

Token Attributes

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
1	Name attribute	ERC20 <code>name</code>	Public (<code>view</code>)		y	Public (<code>view</code>), in both contexts	<code>name()</code> and <code>private_get_name()</code> . Stored as a <code>PublicImmutable<FieldCompressedString></code> set in the constructor; a <code>PublicImmutable</code> is readable from private functions, which is why a private variant exists. Not mutable post-deployment.
2	Reference to legally required documentation	<code>terms</code>	Public (<code>view</code>)		y	Read public (<code>view</code>); write <code>EXTRA_INFORMATION_ROLE</code>	<code>set_terms(DocumentInfo)</code> and <code>terms()</code> . Mirrors the CMTAT Solidity notation: <code>DocumentInfo</code> carries <code>(name, uri, documentHash)</code> and the contract stamps <code>lastModified</code> from the block timestamp, so <code>terms()</code> returns the equivalent of <code>CMTATTerms {name, doc{uri, documentHash, lastModified}}</code> . <code>name</code> and <code>uri</code> are <code>FieldCompressedString</code> (31 characters each); the <code>bytes32</code> hash is stored as two <code>u128</code> halves, because a Noir <code>Field</code> holds ~254 bits and a 256-bit digest would not fit in one.
3	Decimals (no fractions by default)	ERC20 <code>decimals</code>	Public (<code>view</code>)	- Decimals MUST be set to zero unless governing law permits fractions. - The value MUST be readable, since a holder cannot interpret a balance without it. - CMTAT Solidity allows configurable decimals at deployment	y	Public (<code>view</code>), in both contexts	<code>decimals()</code> / <code>private_get_decimals()</code> . <code>PublicImmutable<u8></code> set at deployment, so configurable per issuance as CMTAT Solidity allows.

Note

Attributes are set once, in the `#[external("public")] #[initializer]` constructor, and never change: `name`, `symbol` and `decimals` are `PublicImmutable`.

That type is what makes the twin getters possible. A `PublicMutable` cannot be read from a private function at all: the read would have to go through a public call, which would publish the caller's address and defeat the privacy of whatever private operation needed it. A `PublicImmutable` has no such problem, because once the circuit proves the value was written in the past, it knows it cannot have changed.

So each attribute has a public getter for external observers, and a private getter (`private_get_name`, `private_get_symbol`, `private_get_decimals`) that a private function can call without leaking who is asking.

`name` and `symbol` are `FieldCompressedString`, which packs a string into a single field element and therefore caps them at **31 characters**; the constructor takes them as `str<31>`. `decimals` is a plain `u8` chosen at deployment, so the CMTAT Solidity behaviour of configurable decimals is preserved rather than being fixed at zero.

The **terms** (criterion 2) are the exception to all of that: they are mutable, held in a `PublicMutable<Terms>` in the extra-information module and written by `set_terms` under `EXTRA_INFORMATION_ROLE`, exactly as CMTAT Solidity allows them to be updated after deployment. The notation follows the Solidity one: the setter takes a `DocumentInfo` of `{name, uri, documentHash}` and the contract stamps `lastModified` itself from the block timestamp, so a caller cannot forge it; `terms()` then returns the equivalent of `CMTATTerms`, flattened because Noir gains nothing from the nested struct.

Two chain-level constraints limit how faithfully the document can be recorded:

- **String length.** `name` and `uri` are `FieldCompressedString`, so each is capped at **31 characters** — enough for an IPFS CID, but not for a long HTTPS path, which may have to be shortened or resolved through a redirect.
- **Hash width.** The `bytes32 documentHash` does not fit in one Noir `Field`, which holds about 254 bits, so it is stored as two `u128` halves, `documentHashHigh` and `documentHashLow`. A caller splits the digest as high 16 bytes and low 16 bytes and reassembles it the same way.

Storing the digest in a single `Field` would have truncated it silently, which is the one outcome a document commitment cannot tolerate.

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
4	Ticker symbol attribute	ERC20 <code>symbol</code>	Public (view)	Optional in the CMTA framework, which lists the attribute as "Ticker symbol (optional)".	y	Public (view), in both contexts	<code>symbol()</code> / <code>private_get_symbol()</code> , <code>PublicImmutable<FieldCompressedString></code> set at deployment.
5	Token ID attribute	<code>tokenId</code>	Public (view)	Optional parameter.	y	Read public (view); write <code>EXTRA_INFORMATION_ROLE</code>	<code>set_token_id(FieldCompressedString)</code> and <code>token_id()</code> , in all three variants. As in CMTAT Solidity the value is written even when it equals the current one. Capped at 31 characters by <code>FieldCompressedString</code> , which fits an ISIN with room to spare.

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
6	Version attribute	<code>version()</code> (IERC3643Version, implemented by VersionModule)	Public (view)	Returns the version of the token implementation, for example "3.2.0". In CMTAT Solidity the value is a constant of the contract code: it changes only through a new deployment or an upgrade, and it is not settable at runtime.	y	Public (view)	<code>version()</code> returns a <code>FieldCompressedString</code> , 0.5.0 at the assessed commit, padded to the 31 characters that type requires. As in the CMTAT Solidity <code>VersionModule</code> it is a compile-time constant , not stored state, so it cannot be desynchronised from the deployed code and changes only through a new deployment.

Note

`tokenId` (criterion 5) is present in all three variants, as `set_token_id` / `token_id` on the same extra-information module that carries the terms. It is a `PublicMutable`, not a `PublicImmutable`, because CMTAT Solidity allows it to be changed after deployment; the write is guarded by `EXTRA_INFORMATION_ROLE` and, as in Solidity, happens even when the new value equals the old one.

`version` (criterion 6) is present as a compile-time constant returned by `version()`.

Aztec already identifies deployed code natively, which is why the constant needs justifying. Every deployment is an instance of a **contract class ID**, a hash of the compiled artifact registered on-chain, and since this contract is not upgradeable that ID cannot change under a live address. An observer can therefore always tell *which artifact* is running from chain-native metadata.

What a class ID cannot give is a *semantic* version. It is a hash: it does not order releases, and it matches nothing a reader could look up as a release tag in the repository. The two are complementary, the class ID identifying the artifact and `version()` naming the release it was built from. See [Version](#).

Token module

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
7	Know total supply	ERC20 <code>totalSupply</code>	Public (view)		y	Public (view)	<code>total_supply()</code> reads a <code>PublicMutable<u128></code> . Deliberately public , and updated by the enqueued public half of every mint and burn — see Privacy and Confidentiality .
8	Know balance	ERC20 <code>balanceOf</code>	Public (view)		y	The holder, and the issuer	<code>balance_of_private(owner)</code> , an <code>#[external("utility")]</code> function summing the owner's <code>UIntNote</code> s in their PXE. Not publicly readable — this is the core privacy property of the implementation, and matches the CMTAT framework wording that only the issuer and the holder should know a balance.

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
9	Transfer tokens	ERC20 <code>transfer</code>	Token holder (<code>msg.sender</code>)		y	Token holder, or a delegate holding an authwit	<code>transfer_private_to_private</code> (<code>from</code> , <code>to</code> , <code>amount</code> , <code>authwit_nonce</code>), <code>#</code> <code>[external("private")]</code> . <code>transfer_batch</code> exists but is capped at one recipient per call by protocol limits. Since 0.4.0 the private profile — <code>transfer_private_to_private</code> , <code>mint_to_private</code> , <code>name</code> , <code>symbol</code> , <code>decimals</code> , <code>balance_of_private</code> , <code>total_supply</code> — answers the AIP-20 selectors exactly (pinned by <code>test_aip20_profile.nr</code>); <code>burn</code> deliberately keeps its own selector because it is <code>BURNER_ROLE</code> -gated where AIP-20's <code>burn_private</code> is holder-authorized.
10	Create tokens	<code>mint</code> / <code>batchMint</code>	Role-restricted (issuer/minter authorized)		y	<code>MINTER_ROLE</code>	<code>mint_to_private</code> (<code>to</code> , <code>amount</code>) and <code>mint_batch</code> (<code>accounts</code> , <code>amounts</code>). The role check runs in the enqueued public half (<code>_mint</code>), because roles live in public state. No authwit: only the minter may mint.
11	Cancel tokens	<code>burn</code> / <code>batchBurn</code> / <code>burnFrom</code>	Role-restricted (issuer/burner authorized)	Implementations SHOULD use a dedicated issuer/authorized burn path for forced cancellation scenarios.	y	<code>BURNER_ROLE</code> , plus an authwit from the holder when the caller is not the holder	<code>burn</code> (<code>from</code> , <code>amount</code> , <code>authwit_nonce</code>) and <code>burn_batch</code> (<code>from</code> , <code>amounts</code> , <code>authwit_nonce</code>). The holder's consent is cryptographically required, not merely policy — see criterion 12 and Self-Burn .

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
12	User-approved cancellation	<code>burnFrom</code> (<code>address account</code> , <code>uint256 value</code>)	Role-restricted (<code>BURNER_FROM_ROLE</code>) and an ERC-20 allowance granted by the token holder	The token holder authorizes the cancellation with an <code>approve</code> , and the issuer, or an address it has authorized such as a bridge, performs it. It lets the issuer distinguish a cancellation made to manage supply from one made to carry out a court order. Cancellation by the issuer alone is criterion 11; cancellation by the holder alone is not offered by default — see Self-Burn .	y	<code>BURNER_ROLE</code> and an authwit granted by the holder	This is the <i>only</i> burn path available to the issuer: <code>burn</code> with <code>authwit_nonce != 0</code> requires a witness signed by <code>from</code> . The dual control the criterion describes — role plus holder authorisation — is therefore always enforced here, whereas in CMTAT Solidity criterion 11 offers a path without it.
13	Approve	ERC20 <code>approve</code> (<code>address spender</code> , <code>uint256 value</code>)	Token holder	Grants a delegate permission to transfer a specific amount of tokens from the token account. This is optional, but implementations SHOULD include it since secondary market capability may depend on delegated approval to automate trading and settlement for regulated entities. Issuers SHOULD consult relevant trading and settlement venues if listing is contemplated.	partial	Token holder	Authentication witnesses, not allowances. Single-use, bound to one exact call, nullified on use, revocable with <code>cancel_authwit</code> (<code>inner_hash</code>). No standing per-spender amount. See the note in the Compliance table .

Note

Every value-moving function is split in two halves, and understanding that split explains most of the answers in this document.

The **private half** runs on the holder's own device. It reads the freeze and validation flags, spends the sender's notes and creates the recipient's. Because it is private, it cannot read mutable public state and cannot see the role table.

The **public half** is then enqueued with `self.enqueue_self` and runs on the sequencer after all private execution. It performs the role check and the pause check, and updates `total_supply`. It is marked `#[external("public")] #[only_self]`, so nothing outside the contract can call it. A revert here reverts the whole transaction, which is what makes the role and pause checks binding even though they run after the note work.

Concretely, `mint_to_private` calls `tokenModule::mint_private(...)` (screening and note movement, in the shared library) and then `self.enqueue_self._mint(self.msg_sender(), amount); transfer_private_to_private` and `burn` are built the same way. Note that the caller passed to the public half is the *private* `msg_sender`, so the role is checked against the real user, not the contract.

A balance is the sum of a holder's `UintNote`s. `BalanceSet::add` and `BalanceSet::sub` do not simply write a number: they return a note **message** that must be delivered, and `sub` asserts `Balance too low` if it cannot gather enough notes. Each message is delivered twice — to the note's owner, and to the issuer — which is the mechanism behind criterion 8 and the whole of [Privacy and Confidentiality](#).

Pause module (mandatory)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
14	Pause tokens	<code>pause</code>	Role-restricted (pauser/admin authorized)	Pause must prevent all transfers until <code>unpause</code> is called.	y	<code>PAUSE_ROLE</code>	<code>pause_contract()</code> . Effective immediately, because the flag is a <code>PublicMutable<bool></code> checked in the enqueued public half of transfer. As in CMTAT Solidity, a pause does not stop mint or burn; deactivation does. A revert there reverts the whole transaction.
15	Unpause tokens	<code>unpause</code>	Role-restricted (pauser/admin authorized)		y	<code>PAUSE_ROLE</code>	<code>unpause_contract()</code> . Reverts if the contract is not paused.
16	Know pause status	<code>paused()</code>	Public (<code>view</code>)	Any person MUST be able to determine whether the token is paused; a pause that cannot be read leaves a holder unable to tell why a transfer was refused.	y	Public (<code>view</code>)	<code>public_get_pause()</code> returns <code>1</code> or <code>0</code> .

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
17	Deactivate contract	<code>deactivateContract</code>	Role-restricted (admin authorized)	Must permanently disable the token (except in upgradeability patterns where deactivation behavior is explicitly defined).	y	<code>DEFAULT_ADMIN_ROLE</code>	<code>deactivateContract()</code> . Follows the CMTAT Solidity model: the contract must already be paused, deactivation is refused if it is already deactivated, and <code>unpauseContract</code> refuses to run once the flag is set — which is what makes it permanent. Emits a <code>Deactivated</code> public event carrying the caller. Transfer stops because a deactivated token is paused forever; mint and burn, which survive a pause as in CMTAT Solidity, assert not-deactivated explicitly in their enqueued public half — the same explicit check CMTAT's <code>_canMintBurnByModule</code> makes.
18	Know deactivate status	<code>deactivated()</code>	Public (view)	Any person MUST be able to determine whether the token has been deactivated. In CMTAT Solidity the function is declared by the draft <code>IERC8343</code> interface.	y	Public (view)	<code>public_get_deactivated()</code> returns <code>1</code> or <code>0</code> , readable by anyone exactly as <code>public_get_pause()</code> is.

Note

Pause is the one control in this contract that takes effect **immediately**. Freeze does not, and the difference comes from where each flag is read.

The pause flag is a `PublicMutable<bool>`. It is read only in the public half of mint, transfer and burn, which runs on the sequencer against current public state — so a pause is visible to the very next transaction. Freeze and the validation lists, by contrast, are read in the *private* half, which cannot see current public state at all; they must therefore be `DelayedPublicMutable`, and that is where their delay comes from.

The practical consequence for an operator is that pause is the tool for anything urgent. The repository documents the combination: to freeze an account without leaving a window in which it can still move tokens, pause the token, schedule the freeze, wait out the delay, then unpause.

`pauseContract` reverts if the contract is already paused and `unpauseContract` reverts if it is not, so the state cannot be set redundantly. Both are guarded by `PAUSE_ROLE`. Note the warning inherited from CMTAT: revoking `PAUSE_ROLE` while the contract is paused can leave it stuck.

Deactivation is built on top of the pause, as it is in CMTAT Solidity. `deactivateContract()` requires `DEFAULT_ADMIN_ROLE`, requires the contract to be **already paused**, and refuses to run twice. Once the flag is set, `unpauseContract` refuses to run, so the pause can never be lifted — that refusal is the whole of the permanence guarantee, and it is why the flag itself is never cleared.

Nothing else needed a deactivation check. Because mint, transfer and burn each assert not-paused in their enqueued public half, and a deactivated contract is paused for good, all three are already blocked. CMTAT Solidity reaches the same conclusion for its transfer path and says so in a comment, but has to add an explicit `_requireNotDeactivated()` to mint and burn because *its* mint and burn are permitted while paused. Here they are not, so the pause assertion covers every path.

The one operational consequence worth stating: a deactivated token is indistinguishable from a paused one to any check inside the contract. An observer tells them apart with

`public_get_deactivated()`, and the `Deactivated` event records who did it and when.

Enforcement

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
19	Freeze	<code>freeze</code> or <code>setAddressFrozen(true)</code> (inferred from extracted PDF text)	Role-restricted (compliance/admin authorized)	Must block transfers to and from a given address. Single-function implementations are acceptable if they set a frozen status.	y	ENFORCEMENT_ROLE	<code>freeze(user, FreezableFlag { is_frozen: true })</code> . Blocks both directions: the private internal half of mint, transfer and burn asserts <code>Frozen: Sender / Frozen: Recipient</code> . The flag is a <code>DelayedPublicMutable</code> , so it becomes effective after <code>CHANGE_ROLES_DELAY_SECONDS</code> rather than in the next block — a window during which the target can still move tokens. CMTAT Solidity has the same exposure in a shorter and less deterministic form, since a freeze transaction sits in a public mempool and can be front-run; see the note below.
20	Unfreeze	<code>unfreeze</code> or <code>setAddressFrozen(false)</code> (inferred from extracted PDF text)	Role-restricted (compliance/admin authorized)	Single-function implementations are acceptable if they clear a frozen status.	y	ENFORCEMENT_ROLE	<code>unfreeze(user, FreezableFlag { is_frozen: false })</code> . Same delay. Implemented as two functions rather than one setter, which the template explicitly permits — see Freeze .
21	Know frozen status	<code>isFrozen(address account)</code>	Public (view)	Any person MUST be able to determine whether a given address is frozen. On a ledger providing confidentiality the reading MAY be restricted to the issuer, the holder concerned and the third parties the issuer authorizes — see Privacy and Confidentiality .	y	Public (view)	<code>get_frozen(user)</code> returns 1 or 0. The flag is public even though balances are not; the template allows restricting it on a confidential ledger, but this implementation does not.

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
22	Enforce a transfer	<code>forcedTransfer(address from, address to, uint256 value)</code>	Role-restricted (operator/compliance authorized)	Enforcement transfer is performed via <code>forcedTransfer</code> .	n	—	Cryptographically impossible in this design, not merely unimplemented — see Forced Burn and Forced Transfer .

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
23	Partial freeze	<code>freezePartialTokens(address account, uint256 value) / unfreezePartialTokens(address account, uint256 value)</code>	Role-restricted (operator/compliance authorized)	Intended only to block a sold amount to avoid double-spend during settlement.	n	—	Freezing is all-or-nothing per address. Locking part of a balance would require the contract to reason about note amounts it cannot read.
24	Know active balance	<code>getActiveBalanceOf(address account)</code>	Public (view)	The balance the holder can still transfer, that is the total balance less the partially frozen amount. Only meaningful where partial freeze (criterion 23) is offered.	n	—	Only meaningful with criterion 23.
25	Know frozen balance	<code>getFrozenTokens(address account)</code>	Public (view)	The partially frozen amount held on an address. Declared by the draft IERC7943 interface in CMTAT Solidity. On a ledger providing confidentiality the reading MAY be restricted in the same way as the frozen status (criterion 21).	n	—	Only meaningful with criterion 23.

Note

Freezing is implemented as two guarded functions rather than the ERC-3643 single setter — `freeze(user, FreezableFlag { is_frozen: true })` and `unfreeze(user, ...)` — which the template explicitly permits for non-EVM chains. Each still takes the flag value as an argument, so the pair is closer to two guarded setters than to two verbs; `freeze` asserts the address is not already frozen and `unfreeze` that it is.

The check itself runs in the **private** half of each operation, in the `Screening` implementation of `lib/src/modules/tokenModule.nr` that every variant passes to the shared chains: a mint asserts `Frozen: Recipient`, a transfer asserts both `Frozen: Sender` and `Frozen: Recipient`, and a burn asserts the holder is not frozen (`Frozen: Sender`). Blocking both directions is what distinguishes freeze from a simple spend block.

The flag is a `Map<AztecAddress, DelayedPublicMutable<FreezableFlag, CHANGE_ROLES_DELAY_SECONDS>>`. The delay is not a tuning choice: a private function proves its execution against a historical state, so it can only trust a public value that is guaranteed not to change for a known window. Reading the flag any other way — through a public call — would publish the caller's address on every transfer. The delay is the price of checking compliance state privately.

Why this is answered y and not partial. The delay opens a window in which a target who is watching can still move tokens. That window is not unique to this chain.

- **CMTAT Solidity has the same exposure.** A freeze transaction sits in the public mempool where anyone can see it, and a monitoring target can submit a competing transfer with a higher priority fee and be included first.
- **Its window is not one block either.** It lasts until the freeze is included, which is longer if the transaction is underpriced or the network is congested, and the party building the block decides the order inside it.

- **The feature the criteria ask for is fully present.** A role-restricted flag that blocks transfers in both directions and is publicly readable is exactly what criteria 19 and 20 describe, and the template's own definition of `y` allows the chain-level mechanism to differ.

What differs is whether the issuer can do anything about it.

- **On Ethereum the window can be won, or hidden.** The freezer can often win the race by paying more, and can avoid it altogether by submitting through a private relay such as Flashbots Protect: the transaction is then not in the public mempool, so the target has nothing to react to. The window still exists, but it is no longer observable.
- **Here it can be neither won nor hidden.** It is protocol-enforced rather than a consequence of transaction visibility: the scheduled change is visible in public state and becomes effective only after `CHANGE_ROLES_DELAY_SECONDS`.
- **The mitigation is procedural rather than transactional.** Pause the token, schedule the freeze, wait out the delay, then unpause — which blocks every holder for the duration instead of racing one address.

This is recorded in the technical documentation (`doc/README.md`) under "Limitations", and suggested as an addition to the CMTAT criteria in `cmtat_suggestion.md`.

Because a frozen holding can be neither transferred nor burned (there is no forced path), freezing is terminal for that position until it is unfrozen. The accounting remedy the repository documents is to write the holding off against the public total supply, which the issuer can compute from its note copies.

Transfer restriction (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
26	Conditional transfer request	<code>RuleConditionalTransferRight.detectTransferRestriction(from, to, value) / detectTransferRestrictionFrom(sender, from, to, value) and approvedCount(from, to, value)</code>	Public (view)	Request is represented by a transfer restricted until approval count is non-zero.	n	—	No per-transfer approval workflow.
27	Conditional transfer approval	<code>RuleConditionalTransferRight.approveTransfer(from, to, value) (or approveAndTransferIfAllowed)</code>	Role-restricted (compliance/approver authorized)	Approval is consumed on transfer via <code>transferred(...)</code> ; cancellation via <code>cancelTransferApproval(...)</code> .	n	—	See criterion 26.
28	Assign to whitelist	CMTAT Allowlist: <code>setAddressAllowlist(account, status)</code> , <code>patchAddressAllowlist(accounts, status)</code> , <code>isAllowlisted(account)</code> ; Rules whitelist: <code>addAddress, removeAddress, addAddresses, removeAddresses, isAddressListed</code>	Role-restricted for setters; public (view) for checks	CMTAT Allowlist and Rules whitelist are alternative whitelist implementations.	y	<code>ADDRESS_LIST_ADD_ROLE</code> to add, <code>ADDRESS_LIST_REMOVE_ROLE</code> to remove, <code>VALIDATION_ROLE</code> to switch the mode on; public read	<code>add to list(address, UserFlags) / remove from list(address, UserFlags), with set_operations(SetFlag) choosing which list is enforced and get_operations() reading it back. Blacklist and whitelist share one module; the rule engine of CMTAT Solidity is merged into it rather than being an external contract. List changes carry the same <code>CHANGE_ROLES_DELAY_SECONDS</code> delay as the freeze flag.</code>

Note

Restrictions live **inside the token**, in the validation module, rather than behind an external rule engine: the CMTAT Solidity RuleEngine and the Rules contracts are merged into one module with a single interface, so there is no pluggable hook and no external rule contract to deploy.

`operateOnTransfer(from, to)` reads the `SetFlag` operations switch and dispatches to **exactly one** mode, blacklist first and then whitelist, taking the first that is enabled. The modes therefore do **not** compose: enabling both runs the blacklist only. If no mode is enabled, no check runs and the transfer proceeds.

A rejected transfer **reverts** with a message (`The sender is in the blacklist`, `The recipient is not in the whitelist`, and so on). There is no ERC-1404 restriction code and no non-reverting read path equivalent to `detectTransferRestriction`, so a caller cannot test a transfer before attempting it — a wallet has to simulate the call and interpret the failure.

Both the per-address flags and the operations switch are `DelayedPublicMutable`, so adding an address to a blacklist, or turning a mode on, only bites after the contract's delay setting:

`CHANGE_ROLES_DELAY_SECONDS`, one hour initially and adjustable by the admin. This is the same delay as freeze and it has the same cause; see [Enforcement](#).

There is no sanction-list mode. Earlier revisions declared a third mode whose handler was `panic("not implemented.")`, so enabling it blocked every transfer; the flag has been removed rather than left as a trap. Screening against a sanctions register would need an on-chain list to read, and Aztec has no equivalent of the Chainalysis oracle used on Ethereum.

Access Control

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
29	Grant role	<code>grantRole(bytes32 role, address account)</code> (OpenZeppelin AccessControl via CMTAT/Rules modules)	Role admin (<code>DEFAULT_ADMIN_ROLE</code> or role admin)	Used for roles such as <code>ALLOWLIST_ROLE</code> , <code>DEBT_ROLE</code> , <code>OPERATOR_ROLE</code> , <code>COMPLIANCE_MANAGER_ROLE</code> .	y	<code>DEFAULT_ADMIN_ROLE</code>	<code>grant_role(role, account)</code> . Roles are numeric Field globals rather than bytes32 hashes: <code>DEFAULT_ADMIN_ROLE</code> 1, <code>PAUSE_ROLE</code> 2, <code>ENFORCEMENT_ROLE</code> 3, <code>VALIDATION_ROLE</code> 4, <code>ADDRESS_LIST_ADD_ROLE</code> 5, <code>ADDRESS_LIST_REMOVE_ROLE</code> 6, <code>MINTER_ROLE</code> 7, <code>BURNER_ROLE</code> 8, <code>DEBT_ROLE</code> 9, <code>DEBT_CREDIT_EVENT_ROLE</code> 10, <code>EXTRA_INFORMATION_ROLE</code> 11. Emits a <code>NewRole</code> public event.
30	Revoke role	<code>revokeRole(bytes32 role, address account)</code>	Role admin (<code>DEFAULT_ADMIN_ROLE</code> or role admin)	AccessControl role removal.	y	<code>DEFAULT_ADMIN_ROLE</code>	<code>revoke_role(role, account)</code> . Refuses to revoke from the caller itself. <code>renounce_role(role, callerConfirmation)</code> lets a holder drop its own role.
31	Role attribution	<code>hasRole(bytes32 role, address account) / getRoleAdmin(bytes32 role)</code>	Public (view)	In CMTAT <code>AccessControlModule</code> , <code>DEFAULT_ADMIN_ROLE</code> is treated as having all roles in <code>hasRole</code> .	y	Public (view)	<code>has_role(role, account)</code> returns 1 or 0. Two differences from CMTAT Solidity: <code>DEFAULT_ADMIN_ROLE</code> is not treated as implicitly holding every role, so <code>has_role</code> is an exact lookup; and <code>getRoleAdmin</code> is internal, returning <code>DEFAULT_ADMIN_ROLE</code> for every role, so the admin of a role cannot be changed. Note that because <code>DEFAULT_ADMIN_ROLE</code> administers itself, an admin can appoint another admin.

Note

Roles are numeric Field globals, not bytes32 hashes: `DEFAULT_ADMIN_ROLE` is 1, up to `EXTRA_INFORMATION_ROLE` at 11. The table is `Map<Field, Map<AztecAddress, PublicMutable<bool>>>`, so it lives entirely in **public** state and any observer can enumerate who holds what.

That public placement is what forces the two-phase design described under [Token module](#): a private function cannot read the role table, so every role check is performed in the enqueued public half. `_mint` calls `only_role(MINTER_ROLE, caller)` and `_burn` calls `only_role(BURNER_ROLE, caller)`, in both cases against the address that initiated the private call.

Two differences from CMTAT Solidity are worth recording:

- `has_role` is an **exact lookup**. CMTAT Solidity's `AccessControlModule` treats `DEFAULT_ADMIN_ROLE` as implicitly holding every role; here an admin that has not been

granted `MINTER_ROLE` cannot mint.

- `getRoleAdmin` is internal and returns `DEFAULT_ADMIN_ROLE` for **every** role, so the admin of a role cannot be reassigned. Since `DEFAULT_ADMIN_ROLE` therefore administers itself, an admin can grant `DEFAULT_ADMIN_ROLE` to another account, revoke another admin, or step down with `renounce_role`: the admin set is not fixed. What is fixed is the administering role itself.

`revoke_role` refuses to revoke from the caller itself, so an admin cannot accidentally strip its own rights; `renounce_role(role, callerConfirmation)` is the deliberate path for that, and requires the caller to name itself.

Snapshot (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
32	Schedule a snapshot	<code>scheduleSnapshot(uint256 time)</code>	Role-restricted (snapshot scheduler/admin authorized)	SnapshotEngine <code>ISnapshotScheduler</code> .	n	—	Module absent. See the note in the Compliance table for why a snapshot is structurally hard here, not merely missing.
33	Reschedule a snapshot	<code>rescheduleSnapshot(uint256 oldTime, uint256 newTime)</code>	Role-restricted (snapshot scheduler/admin authorized)	<code>newTime</code> must stay between adjacent scheduled snapshots (not before previous / not after next).	n	—	Module absent.
34	Unschedule a snapshot	<code>unscheduleLastSnapshot(uint256 time) / unscheduleSnapshotNotOptimized(uint256 time)</code>	Role-restricted (snapshot scheduler/admin authorized)	<code>unscheduleLastSnapshot</code> is restricted to the latest scheduled snapshot; <code>unscheduleSnapshotNotOptimized</code> supports generic unscheduling.	n	—	Module absent.
35	Snapshot time	<code>getAllSnapshots() / getNextSnapshots()</code>	Public (view)	Returns created snapshot times and pending scheduled times.	n	—	Module absent.
36	Snapshot total supply	<code>snapshotTotalSupply(uint256 time)</code>	Public (view)	<code>ISnapshotState</code> .	n	—	Module absent. Note that the <i>current</i> total supply is public (criterion 7), so a historical supply can be recovered from chain history even without the module.
37	Snapshot balance	<code>snapshotBalanceOf(uint256 time, address tokenHolder)</code>	Public (view)	<code>ISnapshotState</code> (see also <code>snapshotInfo</code>).	n	—	Module absent, and not reconstructable on-chain: balances are notes in holders' PXEs.

Note

The snapshot module is absent, and in this case that is not the same as unimplemented: the contract could not compute a snapshot even if the module were written.

A snapshot needs the contract to know every holder and every balance at a past instant. On this design it knows neither. A balance is a set of `UintsNote`s living in the holder's own PXE; the chain stores only note *hashes* and nullifiers, which reveal that some note was created or spent but not by whom or for how much. There is no vantage point from which the contract can enumerate holders, and no historical query that would return a balance.

What *is* reconstructable is the issuer's view. Because the issuer receives a copy of every note message ever created (see [Privacy and Confidentiality](#)), it can rebuild the full ledger off-chain and compute a snapshot at any block — provided it has captured those messages, which are delivered off-chain. A corporate action built on snapshots would therefore be an issuer-side process anchored to the public total supply, not an on-chain module.

Dividend (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
38	Distribution create parameters				n	—	Module absent.
39	Distribution set eligibility				n	—	Module absent.
40	Distribution set deposit				n	—	Module absent.
41	Distribution claim deposit				n	—	Module absent.
42	Distribution schedule				n	—	Module absent.
43	Distribution unschedule				n	—	Module absent.

Note

No distribution or dividend functionality is implemented, and CMTAT Solidity does not define one either — the criteria are placeholders pointing at the [IncomeVault](#) prototype.

Any implementation here would inherit the snapshot problem above: eligibility and pro-rata amounts depend on balances the contract cannot read. A workable design would compute entitlements off-chain from the issuer's note copies and settle them either as a second private token transfer or entirely off-chain, with only a commitment published on-chain.

Credit Events (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
44	Flag as default	<code>setCreditEvents(CreditEvents)</code> → <code>creditEvents().flagDefault</code>	Role-restricted (issuer/compliance/admin authorized)	Managed in <code>ICMTATCreditEvents.CreditEvents</code> .	y	<code>DEBT_CREDIT_EVENT_ROLE</code>	<code>get_credit_events(CreditEventsStruct { flagDefault, flagRedeemed, rating })</code> . Stored as a single <code>PublicMutable<CreditEventsStruct></code> ; read with <code>get_credit_events()</code> , which returns the three fields serialized.
45	Remove default flag	<code>setCreditEvents(CreditEvents)</code> with <code>flagDefault = false</code>	Role-restricted (issuer/compliance/admin authorized)	Same function as ID 44 with a different value.	y	<code>DEBT_CREDIT_EVENT_ROLE</code>	Same entry point with <code>flagDefault::false</code> . As in CMTAT Solidity, the setter writes all three attributes at once, so a caller MUST re-supply the values it wants to keep.
46	Flag as redeemed	<code>setCreditEvents(CreditEvents)</code> → <code>creditEvents().flagRedeemed</code>	Role-restricted (issuer/compliance/admin authorized)	Managed in <code>ICMTATCreditEvents.CreditEvents</code> .	y	<code>DEBT_CREDIT_EVENT_ROLE</code>	Same entry point.
47	Set rating	<code>setCreditEvents(CreditEvents)</code> → <code>creditEvents().rating</code>	Role-restricted (issuer/compliance/admin authorized)	Managed in <code>ICMTATCreditEvents.CreditEvents</code> .	y	<code>DEBT_CREDIT_EVENT_ROLE</code>	Same entry point. <code>rating</code> is a <code>FieldCompressedString</code> , so it is capped at 31 characters.

Note

Credit events are stored as a **single** `PublicMutable<CreditEventsStruct>` holding `flagDefault`, `flagRedeemed` and `rating` together, packed into one storage entry. `set_credit_events` is guarded by `DEBT_CREDIT_EVENT_ROLE` and `get_credit_events()` returns the three fields serialized.

The consequence of storing them as one struct — and it is the same in CMTAT Solidity — is that the setter **replaces all three attributes at once**. A caller that wants to raise the default flag without disturbing the rating MUST re-supply the current rating in the same call; omitting it silently clears it. Criteria 44 and 45 (flag and un-flag default) are therefore the same entry point

with a different value, not two functions.

`rating` is a `FieldCompressedString` and is capped at 31 characters.

The values are entirely public: an observer can read a token's default, redemption and rating status even though they cannot read any balance.

Debt (optional)

The debt record is a single `PublicMutable<DebtInformation>`, and `DebtInformation` mirrors the Solidity `ICMTATDebt` interface exactly: a `DebtIdentifier` (who is involved) followed by a `DebtInstrument` (the terms). It is written by `set_debt(DebtInformation)` or `set_debt_instrument(DebtInstrument)` under `DEBT_ROLE` and read by `get_debt()`. As with credit events, a setter replaces everything it covers. String-typed fields are `FieldCompressedString` and therefore capped at 31 characters each.

ID	Attribute	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
48	Guarantor identifier	<code>debt().debtIdentifier.guarantor</code> (set via <code>setDebt</code>)	Read: public (view); write: role-restricted (setDebt)	Debt module (ICMTATDebt.DebtIdentifier).	Y	Read public; write DEBT_ROLE	<code>DebtIdentifier.guarantor</code> (FieldCompressedString).
49	Debtholder representative identifier	<code>debt().debtIdentifier.debtholder</code> (set via <code>setDebt</code>)	Read: public (view); write: role-restricted (setDebt)	Debt module (ICMTATDebt.DebtIdentifier).	Y	Read public; write DEBT_ROLE	<code>DebtIdentifier.debtholder</code> (FieldCompressedString). Renamed from <code>bondHolder</code> to match the Solidity field.
50	Unique identifier / hash	<code>tokenId()</code> and <code>terms().doc.documentHash</code>	Public (view)	<code>tokenId</code> is optional (implementations MAY omit it); document hash is in <code>terms</code> metadata.	Y	Read public (view); write EXTRA_INFORMATION_ROLE	Both halves are now present: <code>token_id()</code> (criterion 5) and the terms document hash, <code>terms().documentHashHigh / documentHashLow</code> (criterion 2).
51	Issuance date	<code>debt().debtInstrument.issuanceDate</code> (set via <code>setDebt</code> / <code>setDebtInstrument</code>)	Read: public (view); write: role-restricted (setDebt*)	Debt module (ICMTATDebt.DebtInstrument).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.issuanceDate</code> (FieldCompressedString).
52	Currency of payments	<code>debt().debtInstrument.currency</code> / <code>debt().debtInstrument.currencyContract</code>	Read: public (view); write: role-restricted (setDebt*)	Supports symbol-like string and token/asset contract address.	Y	Read public; write DEBT_ROLE	Both halves: <code>DebtInstrument.currency</code> (FieldCompressedString) and <code>DebtInstrument.currencyContract</code> (AztecAddress). The address can only name a contract on Aztec, so a payment currency on another ledger has to be identified through the string.
53	Par value	<code>debt().debtInstrument.parValue</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (uint256).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.parValue</code> (Field).
54	Minimum denomination	<code>debt().debtInstrument.minimumDenomination</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (uint256).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.minimumDenomination</code> (Field).
55	Maturity date	<code>debt().debtInstrument.maturityDate</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (string).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.maturityDate</code> (FieldCompressedString).
56	Interest rate	<code>debt().debtInstrument.interestRate</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (uint256).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.interestRate</code> (Field).
57	Coupon payment frequency	<code>debt().debtInstrument.couponPaymentFrequency</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (string).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.couponPaymentFrequency</code> (FieldCompressedString). Renamed from <code>couponFrequency</code> to match the Solidity field.
58	Interest schedule format: A) start date/end date/period; B) start date/end date/day of period; C) date 1/date 2/date 3	<code>debt().debtInstrument.interestScheduleFormat</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (string).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.interestScheduleFormat</code> (FieldCompressedString).
59	Interest payment date: A) period; B) specific date	<code>debt().debtInstrument.interestPaymentDate</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (string).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.interestPaymentDate</code> (FieldCompressedString).
60	Day count convention	<code>debt().debtInstrument.dayCountConvention</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (string).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.dayCountConvention</code> (FieldCompressedString).
61	Business day convention	<code>debt().debtInstrument.businessDayConvention</code>	Read: public (view); write: role-restricted (setDebt*)	Debt module (string).	Y	Read public; write DEBT_ROLE	<code>DebtInstrument.businessDayConvention</code> (FieldCompressedString).

Note

The debt record lives in one `PublicMutable<DebtInformation>`, read by `get_debt()`, which returns the sixteen attributes serialized: the four `DebtIdentifier` fields first, then the twelve `DebtInstrument` fields, in the order of the Solidity structs.

Two setters, as in CMTAT Solidity.

- `set_debt(DebtInformation)` replaces the whole record, identifier and instrument together.
- `set_debt_instrument(DebtInstrument)` replaces only the terms and leaves the identifier as it is — the common case, since the guarantor and the debtholder representative rarely change when a coupon schedule does.
- Both are guarded by `DEBT_ROLE`, and each **replaces everything it covers**: a partial update requires re-supplying every field that must be preserved.

All fourteen criteria now map to a field directly. Criterion 50 (unique identifier / hash) is answered from outside this module, by `token_id()` (criterion 5) and the terms document hash (criterion 2).

Two constraints apply to the values themselves.

- Every string-typed attribute is a `FieldCompressedString`, capped at **31 characters**, where CMTAT Solidity uses an unbounded `string`. That is a real limit for fields such as `interestScheduleFormat` or `businessDayConvention`; where 31 characters is not enough, the value has to be a code or a reference resolved off-chain.
- `interestRate`, `parValue` and `minimumDenomination` are `Field`, not `uint256`. A `Field` holds ~254 bits, so it carries any realistic value, but it is not a 256-bit integer and does not wrap like one.

The whole record is **public state**: anyone can read the debt terms of a deployed token. That is the same visibility as CMTAT Solidity, and deliberate — the terms of an instrument are not what this contract keeps private; balances are.

Guideline sections

The tables in this part of the template are **outside the equivalency count**.

Freeze

The template permits non-EVM implementations to split the ERC-3643 single setter into two functions. This implementation takes that option: `freeze(user, value)` and `unfreeze(user, value)`, each guarded by `ENFORCEMENT_ROLE`. Both still take the flag value as an argument, so the pair is closer to two guarded setters than to two verbs; `freeze` additionally asserts the address is not already frozen, and `unfreeze` that it is.

Note

Splitting the ERC-3643 setter in two is a readability choice, not a functional one: `freeze` and `unfreeze` still take the flag value as an argument, so they are two guarded setters rather than two verbs. What the split buys is a distinct precondition on each — `freeze` asserts the address is not already frozen, `unfreeze` that it is — so a redundant call fails loudly instead of silently rewriting the same value and paying for a scheduled change that changes nothing.

Restriction (optional)

Of the CMTAT Solidity rule catalogue, this implementation offers only list membership, and it lives **inside the token** rather than behind an external rule engine.

Restriction	CMTAT Solidity rule	Present in implementation being approved (y/partial/n)	Implementation details
Whitelist	<code>RuleWhitelist</code>	y	<code>UserFlags.is_whitelisted</code> , enforced on both parties of a transfer when <code>SetFlag.operate_whitelist</code> is on.
Aggregated whitelists	<code>RuleWhitelistWrapper</code>	n	—
Receiver whitelist	<code>RuleReceiverWhitelist</code>	n	Both parties are always checked; the receiver cannot be screened alone.
Spender whitelist	<code>RuleSpenderWhitelist</code>	n	The authwit delegate is not screened.
Blacklist	<code>RuleBlacklist</code>	y	<code>UserFlags.is_blacklisted</code> blocks a listed sender or receiver on transfer, a listed recipient on mint and a listed account on burn — the same three targets as <code>RuleBlacklist</code> .
Sanctions list	<code>RuleSanctionsList</code>	n	No sanction-list mode. <code>RuleSanctionsList</code> reads an on-chain oracle (Chainalysis on Ethereum); Aztec has no equivalent to read from, so a listed address must be blocked through the blacklist instead.
Whitelist and frozen list (ERC-2980)	<code>RuleERC2980</code>	n	Freeze and lists are separate mechanisms here.
Identity registry	<code>RuleIdentityRegistry</code>	n	—
Maximum total supply	<code>RuleMaxTotalSupply</code>	n	No supply cap.
Reserve-backed supply cap	<code>RuleChainlinkPoR</code>	n	No oracle integration.
Maximum balance per address	<code>RuleMaxBalance</code>	n	Not expressible: the contract cannot read a holder's balance.
Conditional transfer	<code>RuleConditionalTransferLight</code>	n	Criteria 26–27.

Restriction	CMTAT Solidity rule	Present in implementation being approved (y/partial/n)	Implementation details
Per-minter quota	<code>RuleMintAllowance</code>	n	—

Order and failure mode. `operateOnTransfer` evaluates exactly one mode per transfer, blacklist first and then whitelist, taking the first that is enabled — they do not compose. If no mode is enabled, no check runs. A rejected transfer **reverts**; there is no ERC-1404-style restriction code and no non-reverting read path equivalent to `detectTransferRestriction`, so a caller cannot test a transfer before attempting it. Because the flags are `DelayedPublicMutable`, a newly listed address is only screened after the delay.

Note

The whole restriction surface is one module inside the token, so the questions the template asks about *where* the logic lives and *in what order* it runs have short answers: it lives in `validationModule.nr`, and exactly one mode runs per transfer.

Two behaviours affect an integrator directly. First, a rejection is a **revert with a message**, not a status code — there is no `detectTransferRestriction` equivalent, so a wallet must simulate the call and read the failure rather than querying first. Second, mint and burn **are** screened by the lists, on their target: `operateOnMint(to)` runs in `_mint_internal` and `operateOnBurn(account)` in `_burn_internal`, mirroring CMTAT Solidity's `_canMintByModuleAndRevert(to)` and `_canBurnByModuleAndRevert(from)`. A blacklisted address can neither be issued to nor redeemed from, which is what the lists are relied on for at issuance.

Earlier revisions of this assessment recorded a documentation mismatch here — the NatSpec on `mint_to_private` claimed a list check the code did not perform. It was resolved by making the code do what the comment said.

On external data sources the template asks about: there are none. Every list is local contract state, so there is no oracle or registry that could be unset, and therefore no fail-open path. That is also why there is no sanctions-list mode: `RuleSanctionsList` exists on Ethereum because a Chainalysis oracle can be queried, and Aztec offers nothing to query.

Version

Implemented, taking the template's **first** option: a constant returned by a read-only entry point, as in CMTAT Solidity. `version()` returns a `FieldCompressedString` holding `0.5.0`, padded to the 31 characters that type requires; the value is a Noir `global`, so it lives in the compiled code rather than in storage.

The template's third option — a state variable restricted to an administrator role — was deliberately not taken. It carries the requirement that the value "cannot be desynchronized from the deployed code", and a compile-time constant satisfies that by construction: there is no setter to call and no storage slot to write, so the only way to change the version is to deploy new code.

Aztec also identifies deployed code natively, through the **contract class ID**: a hash of the compiled artifact registered on-chain, of which each deployment is an instance. Since this contract is not upgradeable, that class ID cannot change under a live address, so the binding between an address and its artifact is permanent. That is the template's second option, and it is available here whether or not `version()` exists.

Note

The class ID alone would not have satisfied the criterion, and the reason is worth recording because it is easy to assume it would.

- **A class ID is a hash, so it does not order releases.** Two artifacts are either equal or unequal; nothing tells a reader which is newer.
- **It does not correspond to anything published.** A reader cannot match a class ID against a release tag in the repository, or against the version named in a changelog or an audit report, without being handed a mapping out of band.
- **It changes for reasons a version should not.** Any recompilation that alters the artifact — a toolchain bump, a comment-only edit that shifts the bytecode — produces a different class ID for the same release.

So the two answer different questions and are both worth having: the class ID identifies *which artifact* is deployed, `version()` names *which release* it was built from. The cost of the latter is that it is now a value someone has to remember to bump; the pre-release checklist in `CHANGELOG.md` carries that step, and a test asserts the constant is non-empty so a blank one fails the suite rather than shipping.

The 31-character cap of `FieldCompressedString` applies here as everywhere else in this contract. It is ample for a semantic version, but a build-metadata suffix of any length would not fit.

CMTAT Extended

CMTAT Functionalities	CMTAT Solidity corresponding features	Present in implementation being approved (y/partial/n)	Implementation details
On-chain snapshot	<code>snapshotModule</code> and <code>snapshotEngine</code>	n	Criteria 32–37.
Forced transfer	<code>forcedTransfer</code>	n	Criterion 22; impossible by construction.
Forced burn	<code>forcedBurn</code>	n	Burning always requires the holder's authwit.
Freeze partial token	<code>freezePartialTokens</code>	n	Criteria 23–25.

CMTAT Functionalities	CMTAT Solidity corresponding features	Present in implementation being approved (y/partial/n)	Implementation details
Integrated whitelisting/allowlisting	CMTAT Allowlist	y	The validation module is integrated in the token, not an external contract.
External whitelisting/allowlisting	CMTAT with rule whitelist	n	No external rule contract is supported.
RuleEngine / transfer hook	CMTAT with RuleEngine	n	Merged into the validation module; there is no pluggable hook.
Upgradeability	CMTAT Upgradeable version	n	Not implemented. A change of contract logic means a new deployment and a migration of holders — and because private balances are notes in each holder's PXE rather than contract storage, that migration cannot be performed by the issuer alone.

CMTAT Functionalities	CMTAT Solidity corresponding features	Present in implementation being approved (y/partial/n)	Implementation details
Fee payer / gasless	CMTAT with ERC-2771 module	n.a.	The criterion asks for a meta-transaction module inside the token; on Aztec such a module would have no work to do. Fee abstraction is a protocol feature: a transaction names its payment method and a Fee Payment Contract settles the fee for the sender, which is how the repository's scripts and tests let freshly created accounts transact. The token carries no gasless logic because none is needed.

Note

Two entries in the table above need the chain-level explanation the template asks for.

Upgradeability. There is no proxy and no native upgrade path in use. That is more consequential here than on an EVM chain: a redeployment cannot carry balances across, because balances are not contract storage the issuer can read and rewrite — they are notes in each holder's PXE, spendable only with each holder's own key. Migrating a live token would require every holder to participate, or the issuer to burn and reissue with each holder's authwit. Any change to the storage layout or the note layout therefore has a real operational cost, which is why the repository treats such changes as MAJOR in its versioning policy.

Gasless / fee payer. This row is answered `n.a.`, the one answer in this assessment outside the template's `y/partial/n` vocabulary. The criterion asks whether the implementation carries CMTAT's ERC-2771 meta-transaction module, and on Aztec that question does not arise: **fee abstraction** is part of the protocol. A transaction names a payment method, and a Fee Payment Contract settles the fee on the sender's behalf — which is how the repository's scripts and end-to-

end tests let freshly created accounts transact without being funded first. Earlier revisions answered `partial`, which understated the position: nothing about the capability is partial, and what is absent is a module that would be redundant. The difference from ERC-2771 is that the payer is chosen **per transaction by the sender**, not configured in the token and not trusted by it — so the token carries no forwarder address, and there is no `_msgSender()` spoofing surface of the kind ERC-2771 introduces.

This row sits in the Extended-functionalities table, not among the 61 numbered criteria, so the change does not affect the scorecard counts above.

Forced Burn and Forced Transfer

Neither is available, and no role or contract change could make them available.

A private balance is a set of notes. Spending a note means publishing its **nullifier**, which is derived from the note and its owner's nullifying key. The issuer does not hold that key, so it cannot nullify a holder's notes — no role, and no contract logic, can grant that ability.

`forcedTransfer` and `forcedBurn` are therefore not implementable in this design, and `burn` always requires an authwit from the holder.

The compensating measure the repository documents is **freeze**: an issuer that must immobilise a position freezes the address, which blocks transfers in both directions after the delay. Where the tokens must also be removed from circulation, the README notes that a permanently frozen holding can be written off by reducing the public total supply, since the issuer knows the holder's balance from its note copies — an accounting remedy, not a transfer.

The repository also records the theoretical escape: if the token were implemented at the account-contract level and the issuer held a shared nullifier for the account holding these notes, forced operations would become possible. That is a different trust model and is not implemented.

Note

The constraint is not a missing feature but a property of the proof system.

Spending a note means publishing its **nullifier**, and the nullifier is derived from the note together with its owner's nullifying key. The issuer does not hold that key. No role, no admin privilege and no contract logic can substitute for it: the constraint is enforced by the protocol's proof system, not by this contract's access control. `forcedTransfer` and `forcedBurn` are therefore not merely unimplemented — they cannot be implemented in this design.

This is where the implementation departs furthest from CMTAT: an issuer facing a court order to move or cancel a holding cannot execute it on-chain, which is a recovery power a security token is normally expected to carry. The remedies available are: **freeze** the address, which immobilises the position in both directions after the delay; and, where the tokens must leave circulation, write the holding off by reducing the public `total_supply`, which the issuer can compute from its note copies. The second is an accounting act, not a transfer — the holder's notes still exist and would still be spendable if the address were ever unfrozen.

The repository records the design that would restore the capability: implementing the token at the account-contract level with the issuer holding a shared nullifier for the account that holds these notes. That is a materially different trust model — the issuer would gain the ability to spend holders' notes — and it is not implemented.

Implementation Details

Functionalities	CMTAT Solidity	Present in implementation being approved (y/partial/n)	Access Control (implementation being approved)	Implementation details
Mint while pause	✓	n	—	Differs from CMTAT Solidity. <code>_mint</code> asserts the contract is not paused, so a pause blocks minting too.
Burn while pause	✓	n	—	Differs from CMTAT Solidity. <code>_burn</code> carries the same assertion.
Self-Burn for everyone	✗	n	—	A holder cannot burn unilaterally: <code>burn</code> also requires <code>BURNER_ROLE</code> on the caller.
Self-Burn for authorized addresses	✓	y	<code>BURNER_ROLE</code>	A holder that also holds <code>BURNER_ROLE</code> burns its own tokens with <code>authwit_nonce = 0</code> .
Standard burn on a frozen address	✗	n	—	<code>tokenModule::burn_private</code> asserts the address is not frozen, and there is no forced path — so a frozen holding cannot be cancelled at all. This is a stricter position than CMTAT Solidity, which offers <code>forcedBurn</code> for exactly this case.
Burn tokens with <code>forcedTransfer</code>	✓	n	—	No <code>forcedTransfer</code> .

Note

One row differs from CMTAT Solidity, and one that used to differ no longer does.

Mint and burn continue through a pause, and stop at deactivation — the CMTAT Solidity behaviour, matched deliberately. CMTAT's `_canMintBurnByModule` checks deactivation and the freeze flag, never `paused()`; only standard transfers consult the pause. Here `_transfer` asserts not-paused while `_mint` and `_burn` assert not-deactivated, so an issuer can issue into and redeem from a paused token, as in the reference. Earlier revisions of this implementation blocked all three during a pause; that was a consequence of where the check sat, not a policy, and it has been aligned.

A frozen holding cannot be cancelled at all. `tokenModule::burn_private` asserts the address is not frozen, and there is no forced path to bypass it. CMTAT Solidity reaches this case with `forcedBurn`; this implementation has no equivalent, so freezing an address and then needing to remove its tokens from circulation leaves only the total-supply write-off described above.

Self-Burn

Self-burn in the CMTA sense — the holder cancelling alone — is **not** offered. `burn` requires `BURNER_ROLE`, so a holder without that role cannot burn even its own tokens, matching CMTAT Solidity's default and the legal reasoning behind it.

What the implementation does guarantee, and CMTAT Solidity does not, is the converse: the issuer cannot burn **without** the holder, because the holder's authwit is cryptographically required. Every cancellation is therefore jointly authorised. An issuer that needs unilateral cancellation for a court order cannot obtain it in this design; see [Forced Burn and Forced Transfer](#).

Cross-Chain Bridge Support

Not supported, and outside the equivalency count. The token has no bridge role, no dedicated cross-chain mint or burn entry point, and no ERC-7802 or CCIP equivalent. `mint_to_private` and `burn` are not reused for a bridge path, so the template's warning about bridge operations bypassing the pause check does not apply.

Aztec's own L1↔L2 messaging exists at protocol level and could carry a burn-and-mint arrangement, but nothing in this contract uses it.

Note

Bridging is not supported and, unlike forced transfer, nothing in the chain's design prevents it — it is simply out of scope for this prototype.

The template's main warning does not apply here: it concerns implementations that reuse the standard `mint` and `burn` for bridge operations and thereby let tokens move across chains while transfers are frozen. This contract exposes no bridge role and no bridge path at all, so `mint_to_private` and `burn` are reachable only by `MINTER_ROLE` and `BURNER_ROLE` under the ordinary checks, and both are blocked while paused.

Were bridging added, the chain-native primitive would be Aztec's L1↔L2 messaging rather than ERC-7802 or CCIP, neither of which is applicable off the EVM. The burn-and-mint model itself would carry over, but the authwit requirement on `burn` would need addressing: a bridge cannot burn a holder's tokens without that holder's witness, which is the same constraint that rules out forced burn.

Privacy and Confidentiality

Aztec is a privacy L2: a private function runs on the user's own device (in the **PXE**), and the network verifies a zero-knowledge proof of it. State is split in two. Private state is a set of **notes** — here `UintNote`s holding a `u128` amount — whose hashes are published on-chain while their contents are not; spending one publishes a **nullifier** derived from the note and its owner's key. Public state is an ordinary key-value store readable by anyone.

This implementation deliberately keeps compliance state public and holdings private.

Two tables answer the template's question from different angles. The first is per **operation**, which is the view needed when checking one criterion: what a caller discloses by invoking that entry point. The second is per **data item**, which is the view the template asks for.

What each operation publishes

Operation	Published on chain	Stays private
<code>mint_to_private</code>	the minter and the amount, as arguments of <code>_mint(caller, amount)</code> ; <code>total_supply</code> rises	the recipient, and the note itself
<code>transfer_private_to_private</code>	only that a transfer of this token occurred: <code>_transfer()</code> takes no arguments	sender, recipient and amount
<code>burn</code>	the burner and the amount, as arguments of <code>_burn(caller, amount)</code> ; <code>total_supply</code> falls	the debited account, unless the burner is the holder
<code>transfer_private_to_public</code>	the public party and the amount, as arguments of <code>_credit_public(to, amount)</code>	the private sender
<code>transfer_public_to_private</code>	the public party and the amount, as arguments of <code>_debit_public(from, amount)</code>	the private recipient
<code>transfer_private_to_commitment</code>	that a transfer occurred, and the amount unencrypted in the completion log, tagged by the commitment	both parties
<code>initialize_transfer_commitment</code>	nothing: the validity commitment is a nullifier	the recipient and the completer

Two disclosures are common to every value-moving operation, whatever the row says. The transaction's **nullifiers and note hashes**, padded so that their count reveals little. And an **expiration timestamp**, because reading a delayed value (the issuer address, the freeze and list flags) bounds how long the transaction may be included. A mint and a burn also move `total_supply`, which is public, so the amount is inferable from the supply delta even where it is not passed as an argument.

Privacy table

Data	Visibility in CMTAT Solidity	Visibility in the implementation being approved	Available to the issuer (<code>y/n</code>)	Other readers	Implementation details
------	------------------------------	---	--	---------------	------------------------

Data	Visibility in CMTAT Solidity	Visibility in the implementation being approved	Available to the issuer (y/n)	Other readers	Implementation details
Balance of an address	<code>public</code>	<code>private</code> , or <code>private</code> and <code>public</code> at the holder's choice when the token is deployed with <code>public_side_enabled = true</code>	<code>y</code>	The holder; anyone, for the public part of a hybrid balance	Notes live in the holder's PXE, and the issuer receives a copy of every note (see below), so it can reconstruct any holder's balance. With the public side enabled a holder may additionally move value to a public balance readable by anyone through <code>balance_of_public</code> ; nothing moves there without that holder's own transaction.
Transfer amount	<code>public</code>	<code>private</code> on the direct path; visible on the private/public bridges and on a commitment completion	<code>y</code>	Sender and recipient; anyone, on a bridge or a completion	Carried in encrypted note messages on <code>transfer_private_to_private</code> , never in public calldata. The bridges publish the amount and the public side's party by design, and a commitment completion emits <code>[storage_slot, value]</code> unencrypted, tagged by the commitment: the value is readable, the recipient is not. Both require <code>public_side_enabled = true</code> and a holder who chooses that path.
Transfer participants	<code>public</code>	<code>private</code>	<code>y</code>	Sender and recipient	The transaction reveals that <i>some</i> transfer occurred and its nullifiers and note hashes, but not who transacted with whom.
Minter and minted amount	<code>public</code>	<code>public</code>	<code>y</code>	Everyone	<code>mint_to_private</code> enqueues <code>_mint(caller, amount)</code> , and a public call's arguments are public. The amount is inferable from the <code>total_supply</code> delta regardless; the caller always holds <code>MINTER_ROLE</code> , which is public state. The recipient is not published.
Burner and burned amount	<code>public</code>	<code>public</code>	<code>y</code>	Everyone	<code>burn</code> enqueues <code>_burn(caller, amount)</code> : same reasoning. The debited account is not published — unless the burner is the holder. Granting <code>BURNER_ROLE</code> to holders makes their self-redemptions fully public ; the role is an issuer role by design.
Total supply	<code>public</code>	<code>public</code>	<code>y</code>	Everyone	<code>PublicMutable<u128></code> , updated by the enqueued public half of mint and burn. A deliberate design decision : it makes the amount of every mint and burn inferable from the public delta, which is accepted so that supply remains auditable by anyone.
Token decimals	<code>public</code>	<code>public</code>	<code>y</code>	Everyone	<code>PublicImmutable<u8></code> . Same for <code>name</code> and <code>symbol</code> .

Data	Visibility in CMTAT Solidity	Visibility in the implementation being approved	Available to the issuer (y/n)	Other readers	Implementation details
Frozen / blacklisted addresses	<code>public</code>	<code>public</code>	<code>y</code>	Everyone	<code>get_frozen</code> is a public view, and the validation flags are public. The template would allow restricting these on a confidential ledger; this implementation does not, so an observer can see that a specific address has been frozen or listed even though it cannot see its balance.
Allowlisted / whitelisted addresses	<code>public</code>	<code>public</code>	<code>y</code>	Everyone	As above, via the validation module's public state.
Roles and role holders	<code>public</code>	<code>public</code>	<code>y</code>	Everyone	<code>has_role</code> is a public view.
Pause status	<code>public</code>	<code>public</code>	<code>y</code>	Everyone	<code>public_get_pause</code> .

The two conditional rows depend on one deployment-time choice. The constructor's last argument, `public_side_enabled`, is a `PublicImmutable<bool>`. With `false` — the fully private token — every bridge entry point reverts before touching state, and both rows read simply `private`. With `true` the four AIP-20 bridges and `balance_of_public` exist, and a holder may choose to expose its own side of a move; no other party can expose it for them. An assessor should record which value the assessed deployment uses.

How the issuer retains visibility

Every note this contract creates is delivered **twice**: once to the note's owner, and once to the address in `issuer_address`, using `deliver_to`. The issuer therefore receives the preimage of every note ever created for any holder and can reconstruct balances and the full transfer history. This is what allows the implementation to answer criterion 8 with the issuer as a reader, and it is the mechanism on which any off-chain snapshot or corporate action would be built.

Three consequences **MUST** be recorded:

- **The issuer's copy is delivered offchain, not onchain.** Aztec documents an onchain constrained copy to an auditor as the supported pattern, and the contract compiles that way, but the PXE cannot process an onchain note message addressed to someone who is not the note's owner — note discovery computes the note's nullifier, which needs the owner's key. The copy is therefore sent with `MessageDelivery::offchain()`. The issuer **must capture these messages as they are produced**; a sender who drops one is not detectable on-chain, and there is no on-chain data availability for the issuer's copy. This is the weakest point of the auditability guarantee and is documented in the repository README.
- **The issuer is a single point of disclosure.** It sees every holding of every holder. There is no per-holder view key, no scoped disclosure to a regulator, and no way for a holder to prove its own balance to a third party without involving the issuer. Granting an auditor or a court a scoped read is listed as future work in the repository.
- **Changing the issuer does not revoke past copies.** `issuer_address` is a `DelayedPublicMutable` and can be changed after the delay, but notes already delivered to

the previous issuer remain readable by it.

- **The `Transfer` event gives the issuer what the note copies cannot: an on-chain, constrained record.** Every transfer's event — sender, recipient, amount — is delivered `onchain_constrained` to the issuer as well as to the recipient. An event has no nullifier and no discovery step, so the limitation that keeps the note copies offchain does not apply; this was verified in the test suite. It means the issuer has data-available, sender-unforgeable evidence of every transfer even though its balance reconstruction still rests on offchain note copies. Mint and burn have no such event yet; their public halves publish the minter or burner and the amount, but not the private party.

Consequences for the CMTAT features

- **Total supply** is auditable by anyone because it is public, at the documented cost of leaking each mint and burn amount.
- **Snapshot and dividend** would have to be computed off-chain by the issuer from its note copies; no on-chain module can enumerate holders.
- **Freeze and list checks** are enforced *without* revealing balances, because the flags are public and are read in private functions through `DelayedPublicMutable` — which is precisely why those reads carry a delay. Making the lists private would remove the delay problem only by moving it, and would require a different construction.
- **Forced transfer** cannot be recovered by any disclosure mechanism: reading a balance is not the same as being able to spend it, and only the owner's key can nullify a note.

Note

How privacy works on this chain. Aztec splits execution in two. A private function runs on the user's own device inside the PXE and produces a zero-knowledge proof; the network verifies the proof without seeing the inputs. Private state is a set of **notes** whose hashes are appended to an on-chain tree while their contents stay off-chain; spending a note publishes a **nullifier** derived from the note and the owner's key, and the protocol rejects duplicates, which is what prevents double-spending. Both trees are append-only, so that spending a note cannot be linked to its creation. Public state is an ordinary key-value store readable by anyone. This implementation puts holdings in the first and compliance state in the second.

How the total supply stays auditable. It is simply public. `total_supply` is a `PublicMutable<u128>` updated by the enqueued public half of every mint and burn, so anyone can read it and its history without any disclosure mechanism. The accepted cost is that the *amount* of each mint and burn is inferable from the public delta — a leak the project takes deliberately, in exchange for supply being verifiable by parties who have no relationship with the issuer.

How freeze and list checks work without revealing balances. The flags are public and are read inside the private half of a transfer through `DelayedPublicMutable`. Nothing about the holder's balance is revealed, and nothing about who is transacting is published — but the check can only be made against a value the circuit can prove is stable, which is where the delay comes from. Making the lists themselves private would not remove that constraint, only relocate it.

What an issuer, auditor or regulator has to do to obtain a disclosure. Today: ask the issuer. The issuer receives a copy of every note message and can reconstruct any holder's history, but there is no scoped credential — no per-holder view key, no decryption share, no observer role — that would let a regulator read one holder's position without the issuer, or let a holder prove its own balance to a third party without the issuer. Adding shareable audit keys is recorded as future work in the repository README.

Privacy does not remove a mandatory capability, with one exception that is not about reading. Every mandatory criterion that requires the issuer to observe a balance is satisfied through the issuer's note copies. The capability the design does remove — forced transfer — is not a reading problem and no disclosure mechanism would restore it: reading a balance and being able to spend it are different powers, and only the owner's key confers the second.

Supplementary features

- **Issuer name and description** — `DebtIdentifier.issuerName` and `DebtIdentifier.issuerDescription`, two debt attributes present in the Solidity `ICMTATDebt.DebtIdentifier` but with no counterpart among criteria 48–61.
- **Authwit revocation** — `cancel_authwit(inner_hash)` lets a holder invalidate a granted authentication witness before it is used, by publishing its nullifier. CMTAT Solidity has no equivalent, because an ERC-20 allowance is revoked by overwriting it.
- **Private reads of token attributes** — `private_get_name`, `private_get_symbol`, `private_get_decimals` and `private_get_issuer` allow a private function to read these values without a public call that would leak the caller's address.
- **Batch entry points** — `mint_batch`, `transfer_batch` and `burn_batch` exist, capped at `MAX_ADDR_PER_CALL = 4` for mint and burn and `MAX_TRANSFER_ADDR_PER_CALL = 2` for transfer. Both caps are **measured**, not derived from the protocol constants: each value was set, the tests adjusted and the full suite run. Transfer's is lower because each recipient costs four constrained deliveries once the `Transfer` event goes to both the recipient and the issuer. The logic is written for larger batches, and raising a cap means re-running the suite at the new value.

Conclusion

Token model and architecture. A short recap of the [Architecture](#) section, which holds the detail. The token is an Aztec contract — `CMTATAztec`, or one of its two sibling variants — in Noir with Aztec.nr v5.2.0; there is no native token primitive on Aztec comparable to ERC-20, so the contract implements the CMTAT functions directly. A holder's balance is not a storage slot but a set of `UintNote`s held in that holder's own PXE, reached through an `Owned<BalanceSet>` and summed by `balance_of_private`, while `total_supply` is a `PublicMutable<u128>` and `name`, `symbol` and `decimals` are `PublicImmutable` values fixed at deployment. Noir has no inheritance, so the CMTAT modules are structs held as fields of one storage struct and every entry point is re-declared in the contract; the contract is **not upgradeable**, and because balances are notes in holders' PXEs a migration cannot be performed by the issuer alone.

Access control. Eleven numeric roles in public state, administered by `DEFAULT_ADMIN_ROLE`, which administers itself and can therefore appoint further admins. Because the role table is public and a private function cannot read mutable public state, every private entry point that needs a role check enqueues a public call that performs it — which is also where the pause check runs. A revert in that public half reverts the whole transaction.

Transfer control flow. A transfer runs in two halves. The private half checks that neither party is frozen, applies the validation module's list check, spends the sender's notes and creates the recipient's, delivering each note message to its owner and a copy to the issuer. The enqueued public half asserts the contract is not paused. Freeze and list flags are `DelayedPublicMutable`, so they are readable from the private half without leaking the caller — at the cost of a delay before any change to them takes effect. Delegated transfers use an authentication witness validated by the `#[authorize_once]` macro, which also nullifies the nonce to prevent replay.

Issuance and cancellation. `mint_to_private` is restricted to `MINTER_ROLE` and takes no authwit. `burn` requires `BURNER_ROLE` **and**, whenever the caller is not the holder, an authwit from the holder — so every cancellation is jointly authorised. Both continue through a pause and stop at deactivation, as in CMTAT Solidity; both are blocked on a frozen address and both are screened by the enabled list on their target. There is no forced transfer and no forced burn.

Data and metadata storage. Credit events and debt attributes are stored on-chain as packed structs in public state, written by role-restricted setters that replace everything they cover. String fields are `FieldCompressedString` and are limited to 31 characters. There is no document module in the ERC-1643 sense, but the extra-information module holds `terms` (name, URI and document hash) and `tokenId`.

Main differences from CMTAT Solidity.

- Balances, transfer amounts and counterparties are private; total supply, compliance flags and roles are public.
- The issuer receives a copy of every note, which is how auditability is preserved — but that copy is delivered off-chain, so its availability depends on the issuer capturing it.
- Forced transfer, forced burn and partial freeze are absent because the issuer cannot nullify another holder's notes. This is the single largest functional gap and it is not closable within this design.
- Freeze and list changes take effect only after a delay, a consequence of how private functions read mutable public state on Aztec.
- Snapshot and dividend modules are absent; a snapshot is not reconstructable on-chain.
- Delegation is a single-use authentication witness rather than a standing allowance.

Known limitations and planned work. Batching is capped at four addresses per call for mint and burn and two for transfer, limits established by measurement against the per-call budgets; the transfer cap is lower because its event is delivered constrained to both the recipient and the issuer. There is no sanction-list mode, for lack of an on-chain register to screen against. Scoped disclosure to an auditor or regulator is recorded in the repository as future work; every state-changing entry point emits a public event. Aztec reached mainnet ("Alpha") but its API still changes substantially between releases, and the live networks ran 5.1.0 against this implementation's 5.2.0 when last checked; this contract was migrated from Aztec 0.63.1 to 5.2.0 as an effectively complete rewrite.

Every mandatory criterion is answered `y`, so under the rule stated in the template the implementation should be considered equivalent to CMTAT: none is answered `n`, and none is answered `partial`. The three mandatory gaps recorded in earlier revisions of this assessment — deactivation, its status, and the reference to legally required documentation — have been implemented on the CMTAT Solidity model. What remains outstanding is not a mandatory criterion at all, but the optional ones that cannot be met in this design: forced transfer, forced burn and partial freeze, because the issuer cannot nullify another holder's notes. A reader should not read "equivalent" as "carries every regulatory recovery power a security token is expected to have", and the implementation has not been audited.

Reference

Item	Repository	Version	Commit
Implementation assessed	https://github.com/CMTA/private-CMTAT-aztec	0.5.0 (unreleased)	3843a36eedd91d2ac83d04e142b1e29536bfb48c

Item	Repository	Version	Commit
Assessment template	https://github.com/CMTA/CMTAT-equivalency-assessment	v0.4.0	bac6380e0d587b4512a0dd8d98ee692cb5658027 — the template declares 0.4.0; the tag is not published yet, so the commit is the reference
Aztec toolchain and aztec-nr	https://github.com/AztecProtocol/aztec-nr	v5.2.0	—

The template's own reference table lists the CMTA Solidity repositories the criteria are mapped against; they are not restated here.